Self-Directed Embedded Roadmap: Durable Hardware Fundamentals

Core filter: every stage below has been true since the 1970s and will still be true in 20 years. Voltage, current, timing, interrupts, buses, memory. No framework-of-the-month. The goal isn't to "finish a course" — it's reaching the point where you can look at a circuit and a datasheet and predict what the chip will do, before you power it.

Built around what you own: an ESP32 dev board, a SunFounder electronics kit, and your M1 Mac. Nothing in Phases 1 to 5 needs a purchase. Every phase maps to parts already in your box.

What you have	What it teaches	Phase
ESP32 DevKit V1 (USB-C, ESP32-D0WD-V3, CP2102 bridge)	GPIO, timers, interrupts, ADC, PWM, UART, FreeRTOS, Wi-Fi, BLE	All
Breadboard, ribbon and loose jumpers	Building every circuit; rows vs rails	1
LEDs (red, green, yellow, blue, white, RGB) and resistor assortment	Ohm's law, current limiting, output pins	1 to 2
Tactile buttons, tilt switch	Digital input, pull-ups, debounce, interrupts	2 to 3
Potentiometers, photoresistor, thermistor	ADC, voltage dividers, sensing	4
Servo, active and passive buzzers	PWM duty cycle, frequency, tones	4
S8050, PN2222A, 2N7000 transistors, 1N4007 and Zener diodes, relay	Driving loads a pin can't, flyback protection	5
74HC595 shift register, 7-segment display	Serial-to-parallel; the same idea as SPI	6
555 timer, capacitors, 4N35 optocoupler	Timing in hardware, RC delay, isolation	Bonus
LCD1602 plus loose I2C backpack	I2C, but only after soldering (no iron yet)	6, later

Not for now: the 9V battery clip and barrel jack (USB from your Mac is all you need), and anything touching wall power. Never switch mains with the relay.

Ground rules (the ones written in blood)

- **USB unplugged while wiring.** Plug in only after you've traced every row.
- **Parts go between a signal or power and GND, never bare wire.** Same row = same point.
- **Every LED gets a resistor** (220 ohm to 1k ohm; never below 100 ohm on a pin).
- **GPIO is 3.3V only.** VIN is 5V. Never wire VIN to a signal pin.
- **Pins to avoid early:** 0, 2, 12, 15 (boot straps), 6 to 11 (onboard flash). 34, 35, 36, 39 are input-only. Safe to start: 4, 5, 13, 14, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33.
- **Test before you trust:** when something fails, isolate with a one-line test sketch (e.g. print `digitalRead`) instead of guessing.

Phase 1 — Electricity and the Breadboard (done)

- Voltage as pressure, current as flow, resistance as the limiter; V = I × R
- A circuit is a loop: source, part, return to GND. Nothing is “powered” without a complete path
- Breadboard: 5-hole rows are one point, the center gap isolates halves, rails run the full length
- Series vs parallel: same current in a chain, same voltage across branches
- Why an LED needs a resistor, and what a short circuit is

Milestone project: wire an external LED from D4 through a 220 ohm resistor to GND, then swap to 1k and 100 ohm and explain the brightness change using Ohm’s law. **Done.**

Phase 2 — Digital I/O (done)

Resource: ESP32 datasheet pin chapter; Arduino reference for `pinMode`

- Output vs input: `pinMode`, `digitalWrite`, `digitalRead`
- Floating inputs and pull-ups: why `INPUT_PULLUP` reads HIGH until the button grounds it
- Contact bounce and debouncing; edge detection instead of level detection
- Strapping pins and why some GPIOs misbehave at boot

Milestone project: button on D13 and press counter with debounce. **Done.** Next: combine it with the D4 LED so the button toggles the light, and explain every wire out loud.

Phase 3 — Time and Events (you’re here)

Resource: ESP32 Technical Reference Manual, GPIO and timer chapters

- Why `delay()` blocks everything, and non-blocking timing with `millis()`
- State machines: “what state am I in, what event moves me” instead of nested ifs
- Interrupts: hardware pauses your code to run a short handler (`attachInterrupt`)
- Keep ISRs tiny: set a flag, do the work in the main loop; `volatile` and why
- Hardware timers vs software timing

Milestone project: blink the D4 LED at one rate with a hardware timer while a button interrupt counts presses on D13, with no `delay()` anywhere, and show the count on the Serial Monitor in real time.

Phase 4 — The Analog World

- ADC: turning a voltage into a number (12-bit, 0 to 4095); noise and averaging
- ESP32 detail: use ADC1 pins (32 to 39); ADC2 conflicts with Wi-Fi
- Voltage dividers: how a photoresistor or thermistor plus a fixed resistor becomes a readable voltage
- PWM: duty cycle vs frequency; dimming an LED, positioning a servo, making tones with a buzzer
- Why PWM looks like analog to your eye but is still digital

Milestone project: potentiometer on an ADC pin dims an LED through PWM; then a photoresistor night-light that turns on in the dark. Print raw ADC values and explain the voltage-divider math behind them.

Phase 5 — Driving Real Loads

- A GPIO pin gives roughly 20 mA comfortably; motors, buzzers and relays need more
- Transistor as a switch (S8050 or PN2222A): base resistor, load on the collector, shared ground
- Inductive loads and flyback: why a 1N4007 diode goes across a relay coil
- Powering the servo or relay from VIN (5V) and why the ESP32 may reset on current spikes
- Common ground when two supplies are involved

Milestone project: switch the relay coil (or an active buzzer) from a GPIO pin through a transistor with a flyback diode, triggered by your button. Low-voltage loads only. Be able to draw the schematic from memory.

Phase 6 — Talking to Other Chips (serial buses)

Resource: Datasheets: 74HC595, then any I2C sensor

- UART: two wires, agreed baud rate, start and stop bits (you already use it through the Serial Monitor)
- Shift register 74HC595: clock, data, latch. This is the core idea behind SPI
- SPI: master, MOSI, MISO, SCK, chip select; fast, simple, no addressing
- I2C: two wires, addresses, pull-up resistors, start and stop conditions
- Reading a datasheet: pinout, timing diagram, register map
- Seeing the signal: a ~$10 logic analyzer plus free PulseView turns bus theory into something visible

Milestone project: drive 8 LEDs (or the 7-segment display) from three pins through the 74HC595, first by toggling the pins yourself (bit-banging), then with hardware SPI. **One purchase:** a BME280 sensor (~$5) is the cheapest way to do real I2C without soldering.

Phase 7 — Memory, Storage and Tasks

Resource: ESP-IDF Programming Guide; FreeRTOS kernel documentation

- Flash vs RAM; what lives where; stack vs heap on a chip with ~320 KB of RAM
- Flash partitions and NVS: how a counter survives a reboot
- FreeRTOS: tasks, priorities, queues, mutexes; why two tasks touching one variable is a race condition
- Moving from Arduino to ESP-IDF in C: `app_main`, `gpio_set_level`, `vTaskDelay`
- Reading the boot log: bootloader, partition table, app start

Milestone project: two FreeRTOS tasks connected by a queue — one reads the button and sends events, the other drives the LED — with the press count stored in NVS so it survives power loss. Build it in ESP-IDF C.

Phase 8 — Wireless, Toward Your Goals

Resource: 802.11 and BLE specifications (overviews first); Wireshark

- Wi-Fi as a peripheral: scan, connect, serve a page from the board
- 802.11 frame types: beacon, probe, authentication, association; channels and signal strength
- BLE: advertising vs connecting, GATT basics, what a device broadcasts about itself
- MAC address randomization and what your own devices leak
- Promiscuous mode: reading raw frames from your own network into Wireshark

Milestone project: scan nearby networks and BLE devices with the board and log them, then capture frames from **your own** network and identify each frame type. Test only devices and networks you own or have written permission to test.

What “done” looks like

You can look at a circuit and a snippet of firmware — on any chip, in any framework — and reason about:

- Where the current flows, what limits it, and what could short or overload a pin
- What state each pin is in at boot and when the button is pressed, before you power it
- What the CPU is doing in time: what blocks, what interrupts, what runs in which task
- What is on the wire between two chips, and how to prove it with a logic analyzer
- What a wireless device is broadcasting, and why

That’s the actual target: the hardware layer under every smart device, tracker and radio, stripped of the parts that don’t age well. Specific boards, libraries and tools sit on top of this and are fast to pick up once it’s solid. It’s also the foundation your security and privacy goals stand on.